Machine Learning IC Design Report HW1

File structure:

```
├── alexnet.cpp
├── Alexnet.h
├── Conv_functions.h
├── data
│   ├── cat.txt
│   ├── conv1_bias.txt
│   ├── conv1_weight.txt
│   ├── conv2_bias.txt
│   ├── conv2_weight.txt
│   ├── conv3_bias.txt
│   ├── conv3_weight.txt
│   ├── conv4_bias.txt
│   ├── conv4_weight.txt
│   ├── conv5_bias.txt
│   ├── conv5_weight.txt
│   ├── dog.txt
│   ├── fc6_bias.txt
│   ├── fc6_weight.txt
│   ├── fc7_bias.txt
│   ├── fc7_weight.txt
│   ├── fc8_bias.txt
│   ├── fc8_weight.txt
│   └── imagenet_classes.txt
├── Layer_param.h
├── Makefile
├── other
│   ├── cat.jpg
│   └── dog.jpg
├── result_cat.log
├── result_dog.log
└── Utils.h
```

Dog result:

```
3. ee26.si2.iee.nycu.edu.tw (mlchip02

4 directories, 39 files
16:27 mlchip021@ee26[~/hw1]% make dog
g++ -I . -I  /RAID2/COURSE/2025_Spring/mlchip/mlchipTA
ux64 -o run *.cpp -lsystemc-2.3.3 -Wl,-rpath,/RAID2/CO
In file included from alexnet.cpp:5:0:
Alexnet.h:27:23: warning: non-static data member initi
     string file_dir = "./data/";
                       ^
./run dog.txt

        SystemC 2.3.3-Accellera --- Mar  4 2025 01:46:
        Copyright (c) 1996-2018 by all Contributors,
        ALL RIGHTS RESERVED
Top 100 classes:
=================================================
  idx |      val | possibility | class name
-------------------------------------------------
  207 |    16.59 |       38.63 | golden retriever
  175 |    15.57 |       13.86 | otterhound
  220 |    15.36 |       11.26 | Sussex spaniel
  163 |    15.00 |        7.86 | bloodhound
  219 |    14.59 |        5.22 | cocker spaniel
  168 |    14.39 |        4.28 | redbone
  160 |    14.35 |        4.07 | Afghan hound
  213 |    14.18 |        3.46 | Irish setter
  291 |    14.10 |        3.19 | lion
  211 |    13.01 |        1.07 | vizsla
  244 |    12.81 |        0.88 | Tibetan mastiff
  216 |    12.69 |        0.78 | clumber
  200 |    12.46 |        0.62 | Tibetan terrier
  159 |    12.42 |        0.59 | Rhodesian ridgeback
  152 |    12.38 |        0.57 | Japanese spaniel
```

Cat result:

```
3. ee26.si2.iee.nycu.edu.tw (mlchip02
rm -rf run
16:27 mlchip021@ee26[~/hw1]% make cat
g++ -I . -I  /RAID2/COURSE/2025_Spring/mlchip/mlch
ux64 -o run *.cpp -lsystemc-2.3.3 -Wl,-rpath,/RAID
In file included from alexnet.cpp:5:0:
Alexnet.h:27:23: warning: non-static data member i
     string file_dir = "./data/";
                       ^
./run cat.txt

        SystemC 2.3.3-Accellera --- Mar  4 2025 01
        Copyright (c) 1996-2018 by all Contributor
        ALL RIGHTS RESERVED
Top 100 classes:
=================================================
  idx |      val | possibility | class name
-------------------------------------------------
  285 |    20.21 |       96.38 | Egyptian cat
  281 |    16.14 |        1.65 | tabby
  282 |    15.73 |        1.10 | tiger cat
  287 |    14.79 |        0.43 | lynx
  728 |    14.41 |        0.29 | plastic bag
  330 |    12.73 |        0.05 | wood rabbit
  331 |    12.19 |        0.03 | hare
  457 |    10.94 |        0.01 | bow tie
  335 |    10.67 |        0.01 | fox squirrel
  463 |    10.57 |        0.01 | bucket
  478 |    10.32 |        0.00 | carton
  876 |    10.29 |        0.00 | tub
  622 |    10.18 |        0.00 | lens cap
  904 |    10.01 |        0.00 | window screen
  700 |     9.56 |        0.00 | paper towel
  278 |     9.39 |        0.00 | kit fox
    8 |     9.29 |        0.00 | hen
  284 |     9.21 |        0.00 | Siamese cat
  722 |     8.80 |        0.00 | ping-pong ball
  434 |     8.77 |        0.00 | bath towel
```

Implementation approach:

Using call function to implement the AlexNet, keep the code short.

The following are the functions.

Convolution

```
vec_3d convolution(const vec_3d& input, const vec_4d& weight, const vec_1d& bias,
    const int in_channel_num, const int out_channel_num, const int kernel_size, const int stride, const int padding){

    int input_height = input[0].size();
    int input_width = input[0][0].size();
    int output_height = (input_height + 2 * padding - kernel_size) / stride + 1;
    int output_width = (input_width + 2 * padding - kernel_size) / stride + 1;

    vec_3d input_padded(in_channel_num, vec_2d(input_height + 2 * padding, vec_1d(input_width + 2 * padding, 0)));
    vec_3d output(out_channel_num, vec_2d(output_height, vec_1d(output_width, 0)));

    for(int c = 0; c < in_channel_num; c++) {
        for(int h = 0; h < input_height; h++) {
            for(int w = 0; w < input_width; w++) {
                input_padded[c][h + padding][w + padding] = input[c][h][w];
            }
        }
    }

    for(int o = 0; o < out_channel_num; o++) {
        for(int h = 0; h < output_height; h++) {
            for(int w = 0; w < output_width; w++) {
                for(int c = 0; c < in_channel_num; c++) {
                    for(int i = 0; i < kernel_size; i++) {
                        for(int j = 0; j < kernel_size; j++) {
                            output[o][h][w] += input_padded[c][h * stride + i][w * stride + j] * weight[o][c][i][j];
                        }
                    }
                }
                output[o][h][w] += bias[o];
            }
        }
    }

    return output;
}
```

Max pool

```
vec_3d max_pooling_3d(const vec_3d& input, const int input_channel_num, const int kernel_size, const int stride) {
    int output_height = (input[0].size() - kernel_size) / stride + 1;
    int output_width = (input[0][0].size() - kernel_size) / stride + 1;

    vec_3d output(input_channel_num, vec_2d(output_height, vec_1d(output_width, 0)));

    for(int c = 0; c < input_channel_num; c++) {
        for(int h = 0; h < output_height; h++) {
            for(int w = 0; w < output_width; w++) {
                float max_val = input[c][h * stride][w * stride];
                for(int i = 0; i < kernel_size; i++) {
                    for(int j = 0; j < kernel_size; j++) {
                        max_val = max(max_val, input[c][h * stride + i][w * stride + j]);
                    }
                }
                output[c][h][w] = max_val;
            }
        }
    }

    return output;
}
```

Relu

```
vec_3d relu_3d(const vec_3d& input) {
    vec_3d output(input.size(), vec_2d(input[0].size(), vec_1d(input[0][0].size(), 0)));

    for(int c = 0; c < input.size(); c++) {
        for(int h = 0; h < input[0].size(); h++) {
            for(int w = 0; w < input[0][0].size(); w++) {
                output[c][h][w] = input[c][h][w] > 0 ? input[c][h][w] : 0;
            }
        }
    }

    return output;
}

vec_1d relu_1d(const vec_1d& input) {
    vec_1d output(input.size(), 0);

    for(int i = 0; i < input.size(); i++) {
        output[i] = input[i] > 0 ? input[i] : 0;
    }

    return output;
}
```

Softmax

```
vec_1d softmax(const vec_1d& input) {
    vec_1d output(input.size(), 0);
    float sum = 0.0;

    for(int i = 0; i < input.size(); i++) {
        output[i] = exp(input[i]);
        sum += output[i];
    }

    for(int i = 0; i < input.size(); i++) {
        output[i] /= sum;
    }

    return output;
}
```

Challenges faced:

This is my first time implementing a large CNN using C++.
During the process, I was not very familiar with the use of vector at the beginning, and the operation of SystemC. I encountered compile problems. Because I could not use C++11, some places were a little different from the teacher's instructions in class, such as pair needs to use "> >" instead of ">>". In addition, some etymological output positioning libraries were used for the first time, so it took some time to understand the syntax. I hope that the teaching assistant's homework instructions will be clearer next time.

Observations:

I found that the padding is somewhat different from the NN I encountered before. Normally, it should be symmetrical padding, but this assignment uses a 2:1 padding method. I don’t know what the benefit is.

Comment:

After this assignment, I feel that I still don’t know what applications SystemC has in hardware. This is because the circuit was too large at the beginning, and I didn’t know how to implement it if I wanted to turn it into a real circuit. For example, it takes more than 20,000 cycles to pass the input data. Later, because I couldn’t realize it, I used software to complete this assignment.

Reference:

Lecture Note
Hw1 handout
ChatGPT